

Self-Contrastive Forward-Forward algorithm

Received: 22 November 2024

Xing Chen¹✉, Dongshu Liu¹, Jérémie Laydevant^{2,3} & Julie Grollier¹✉

Accepted: 11 June 2025

Published online: 01 July 2025

 Check for updates

Agents that operate autonomously benefit from lifelong learning capabilities. However, compatible training algorithms must comply with the decentralized nature of these systems which imposes constraints on both the parameters counts and the computational resources. The Forward-Forward (FF) algorithm is one of these. FF relies only on feedforward operations, the same used for inference, for optimizing layer-wise objectives. This purely forward approach eliminates the need for transpose operations required in traditional back-propagation. Despite its potential, FF has failed to reach state-of-the-art performance on most standard benchmark tasks, in part due to unreliable negative data generation methods for unsupervised learning. In this work, we propose Self-Contrastive Forward-Forward (SCFF) algorithm, a competitive training method aimed at closing this performance gap. Inspired by standard self-supervised contrastive learning for vision tasks, SCFF generates positive and negative inputs applicable across various datasets. The method demonstrates superior performance compared to existing unsupervised local learning algorithms on several benchmark datasets, including MNIST, CIFAR-10, STL-10 and Tiny ImageNet. We extend FF's application to training recurrent neural networks, expanding its utility to sequential data tasks. These findings pave the way for high-accuracy, real-time learning on resource-constrained edge devices.

On-chip edge learning is essential for adapting model weights in response to newly arrived, real-time, unlabeled personal data, making it especially suited for privacy-sensitive applications^{1,2}. Unlike centralized cloud training, edge training also significantly reduces energy consumption, making it a more efficient and sustainable choice^{3,4}.

The surge in progress for inference using in-memory computing systems has yet to be matched in the realm of on-chip learning. Backpropagation⁵, the highest-accuracy algorithm for training neural networks, is particularly challenging to implement in hardware. Its non-locality requires storing all activation functions and their derivatives, and the backward pass necessitates bidirectional crossbar arrays, which doubles the number of transistors or the number of arrays, resulting in higher power and area consumption⁶. Local learning algorithms, which compute weight updates locally within each layer or neuron without relying on dependencies across the entire network, offer a promising solution to these challenges. Thus, there is a pressing need for local algorithms that are not only

efficient for hardware implementations but can also handle time-series data and perform unsupervised learning, all while maintaining high accuracy⁷.

The upper section of Table 1 compares different categories of algorithms designed to address these challenges through their local updates, highlighting their specific advantages and issues for on-chip learning. While many algorithms effectively tackle specific aspects of on-chip learning, none achieve a comprehensive solution that simultaneously resolves all identified challenges.

Direct Feedback alignment (DFA) offers an alternative to back-propagation by simplifying the learning process. Instead of computing exact gradients, it uses fixed, random feedback weights^{8,9}. This approach has been widely adopted in various hardware implementations for on-chip learning^{10–13}. However, DFA still requires a backward pass through one or several synaptic arrays. Its performance also remains limited in deeper networks, particularly those integrating convolutional layers, despite impressive progress on other fronts¹⁴.

¹Laboratoire Albert Fert, CNRS, Thales, Université Paris-Saclay, Palaiseau, France. ²School of Applied and Engineering Physics, Cornell University, Ithaca, NY, USA. ³USRA Research, Institute for Advanced Computer Science, Mountain View, CA, USA. ✉e-mail: xing.chen@cnrs-thales.fr; julie.grollier@cnrs-thales.fr

Table 1 | Comparison of the learning capabilities of local learning methods with Backpropagation (BP) method for online training in crossbar-based hardware and their test accuracy [%] on CIFAR-10, STL-10, and Tiny ImageNet datasets

Algorithms	Local	Unidirectional inference	Unidirectional learning	Simple activation function	Unsupervised	Time series	CIFAR-10	STL-10	Tiny ImageNet	
									Top-1	Top-5
BP	✗	✓	✗(backward pass)	✓(ReLU)	✓	✓	93.8 ⁸⁹	91.70 ⁸⁹	54.8 ⁹⁰	
Energy-based	✓	✗	✗(bidirectional connections)	✗(HardSigmoid)	✗	✗	92.3 ²²	–	–	
Hebb-based	✓	✓	✗(Oja's weight transpose)	✗(Softmax)	✓	✗	80.3 ¹⁶	76.2 ¹⁶	–	37.0 ⁵⁸
Reward Hebb	✓	✓	✗(reward backward)	✓(ReLU)	✓	✓	–	73.6 ¹⁷	–	
DFA	✓	✓	✗(random backward)	✓(Tanh)	✗	✗	73.1 ⁸	–	32.1 ⁹¹	
FF	✓	✓	✓	✓(ReLU)	✗	✗	59.0 ²⁷	–	–	
			✓		✗		59.1 ³³	–	–	
			✓		✗		78.1 ³⁴	–	–	
			✗		✗		84.6 ⁴⁵	–	–	
			✓		✓		60.6 ³⁹	–	–	
SCFF	✓	✓	✓	✓(ReLU)	✓	✓	80.8	77.3	35.7	59.8

For Tiny ImageNet, both the top-1 and top-5 accuracy are shown. The symbols ✓, ✗, and – mean “yes”, “no”, and “no reported results” respectively. “Unidirectional inference” and “Unidirectional learning” means that weight matrices do not need to be transposed during inference or learning. “Unsupervised” means that the algorithm can handle unlabeled data. “Time series” refers to the ability of algorithms to handle time series input with RNN architecture. The evaluation of each criterion is based on the selected reference, which represents a specific implementation chosen for its strong performance on CIFAR-10 or STL-10 benchmarks.

Entries in bold indicate the proposed method and its associated results.

Hebbian-based learning methods¹⁵ present a strong interest for hardware implementations due to their simple two-factor learning rule that only depends on pre- and post- neuron activities. The recent development of algorithms such as SoftHebb¹⁶ has considerably improved the accuracy of these approaches on unsupervised tasks. This success comes however, at the cost of complex-to-implement softmax activation functions. Furthermore, bidirectional crossbar arrays are still needed for learning, as the weight update rule involves the transpose of the weight matrix.

Reward-based Hebbian algorithms incorporate a third factor in the Hebbian learning rule in order to increase the accuracy^{17–19}. The reward signal flows from the output to the intermediate layers through additional weight matrices, thus complicating the implementation.

Energy-based algorithms^{20–23}, such as Equilibrium Propagation^{20,24}, offer spatially local updates, simple activation functions, and accuracies very close to BP. Those models have been implemented in specialized hardware, using architectures like network of variable resistors²⁵ and Ising machines²⁶ to achieve efficient local updates and lower power consumption. However, they require hard-to-implement bidirectional connectivity as well as reaching network equilibrium for each input, which constitutes a hurdle for time-series classification.

Therefore, further work is essential to develop learning algorithms that integrate all the key features necessary for practical implementation: local learning rules, unidirectional inference and learning, simple activation functions, and high accuracy in embedded application tasks, such as unsupervised learning of both static and dynamic data.

Recently, the Forward-Forward (FF) algorithm²⁷ emerged as a promising candidate for hardware-friendly learning due to its simplicity: it is local, unidirectional, works with simple activation functions like ReLU, and supports both static and sequential data. As a result, the FF algorithm has attracted significant interest since its introduction^{28–40}. Early demonstrations of FF in silico already cover a wide range of hardware platforms, including microcontrollers⁴¹, in-memory computing systems utilizing Vertical NAND Flash Memory as synapses⁴², and physical implementations where neurons are replaced by acoustic, optical, or microwave technologies^{43,44}. However, its unsupervised learning performance has stagnated, particularly due to the challenge of generating high-quality negative data.

The lower section of Table 1 highlights recent advancements in the FF algorithm^{27,31,33,34,39,45}, comparing its learning capabilities across

various metrics. Notably, the accuracy of FF supervised training on CIFAR-10 dataset has improved significantly, rising from below 60%²⁷ to 84.7%⁴⁵ by integrating random direct feedback connections, thereby closing the gap with leading supervised, purely local, forward algorithms⁴⁶.

However, the performance of FF has plateaued in areas that are crucial for on-chip learning, such as unsupervised learning and time-series processing. In unsupervised learning, FF has shown only limited success³⁹ and has struggled with datasets more challenging than MNIST⁴⁷, such as CIFAR-10⁴⁸, simplified ImageNet variants like Tiny ImageNet⁴⁹ or the unlabeled dataset STL-10⁵⁰. Additionally, FF is currently unable to effectively handle time-varying sequential data, which significantly limits its applicability to neuromorphic systems that often require processing dynamic, real-world inputs.

These challenges stem from the difficulty of generating “negative” examples that are similar enough to the “positive” training data to provide meaningful contrast, yet distinct enough for the network to learn effective representations. While creating such examples is relatively straightforward in supervised learning, where true or false labels can be integrated into the training process, there is currently no efficient, universal approach for generating positive and negative examples across all datasets. This limitation hinders FF’s effectiveness in unsupervised learning and time-series processing, making it difficult to generalize across different data types and applications.

In this work, we propose the Self-Contrastive Forward-Forward (SCFF) method, which contrasts each data sample against itself to enable efficient learning. Inspired by self-supervised learning, SCFF generates positive and negative examples directly from the data, making FF applicable to unsupervised learning across a wide range of datasets. We demonstrate that SCFF not only extends FF’s capabilities to unsupervised tasks but also surpasses other local algorithms for unsupervised tasks on the MNIST, CIFAR-10, STL-10, and Tiny ImageNet datasets. Moreover, SCFF makes it possible to apply FF to sequential tasks, unlocking its potential for time-series data processing. Specifically, our contributions are as follows:

- We propose the SCFF method, which efficiently generates positive and negative examples to train neural networks using the FF algorithm in an unsupervised manner, applicable across a wide range of datasets.

- We show that SCFF significantly outperforms existing unsupervised local learning algorithms in image classification tasks, achieving test accuracies of $98.70\% \pm 0.01\%$ on MNIST, $80.75\% \pm 0.12\%$ on CIFAR-10, and $77.30\% \pm 0.12\%$ on STL-10 (which includes a small number of labeled examples alongside a much larger set of unlabeled data). SCFF furthermore achieves a top-1 accuracy of $35.67\% \pm 0.42\%$ and a top-5 accuracy of $59.75\% \pm 0.18\%$ on the more complex dataset Tiny ImageNet.
- We present a demonstration of the FF approach being successfully applied to sequential data. Our findings show that the proposed SCFF method effectively learns representations from time-varying sequential data using a recurrent neural network. In the Free Spoken Digit Dataset (FSDD)⁵¹, SCFF training improves accuracy by approximately 10 percentage points compared to reservoir computing methods.
- We conduct a theoretical and illustrative analysis of the distribution of negative examples within the data space. The analysis reveals that negative data points consistently position themselves between distinct clusters of positive examples, which suggests that negative examples play a crucial role in pushing apart and further separating adjacent positive clusters, thereby enhancing the efficiency of classification.

Our results demonstrate the potential of SCFF for efficient, layer-wise learning of meaningful representations in a local, purely forward, and unsupervised manner, enabling online training in hardware environments. In Section “Results”, we introduce SCFF, detailing its hardware applicability, innovative negative example generation method and its training procedure. Next, in section “Discussion”, we will present the results and discuss our findings. Finally, we will explore the relationship of SCFF to other purely forward and/or local learning methods in the discussion of section “Methods”.

Results

The primary innovation of SCFF lies in its method for generating positive and negative examples, making it applicable to both

supervised and unsupervised tasks across a wide variety of datasets. SCFF maintains the hardware-friendly, forward-only nature of FF while significantly enhancing its learning capabilities, particularly for unsupervised tasks and time-series classification.

Creating the negative and positive examples

In the FF algorithm, illustrated in Fig. 1b, positive and negative examples are successively presented to the network input²⁷. A “goodness” score, related to local neuron activity, is computed at each layer, with the training objective being to maximize goodness for positive examples while minimizing it for negative ones. Therefore, the negative examples must be carefully crafted to effectively challenge the network, requiring more sophisticated approaches than basic noise injection or occlusion. For image data, the negative examples should maintain similar short-range correlations to the original data but differ significantly in long-range correlations to produce distinct shapes and categories. For the MNIST dataset, Hinton proposed generating negative examples by applying masks to different digit images and combining them to create hybrid images that look like digits but are not digits (Fig. 1a). While this method works well for MNIST²⁷, it does not easily extend to more complex image databases like CIFAR-10, ImageNet and STL-10, or time series data, resulting in limited accuracy on these benchmarks. Therefore, further development of the FF algorithm must focus on devising robust methods for generating positive and negative examples that are applicable across diverse datasets.

As illustrated in Fig. 1c, contrastive self-supervised learning methods share some similarity with FF. Negative and positive pairs are defined, where a positive pair consists of two different augmented views of the same data sample, and a negative pair of two different samples. Contrastive losses are employed to define the similarity in feature space between each image of a pair, with the training objective being to maximize similarity for positive pairs while minimizing it for negative ones.

In SCFF, we propose to directly take pairs of positive and negative images as inputs to the neural network (Fig. 1d) instead of contrasting

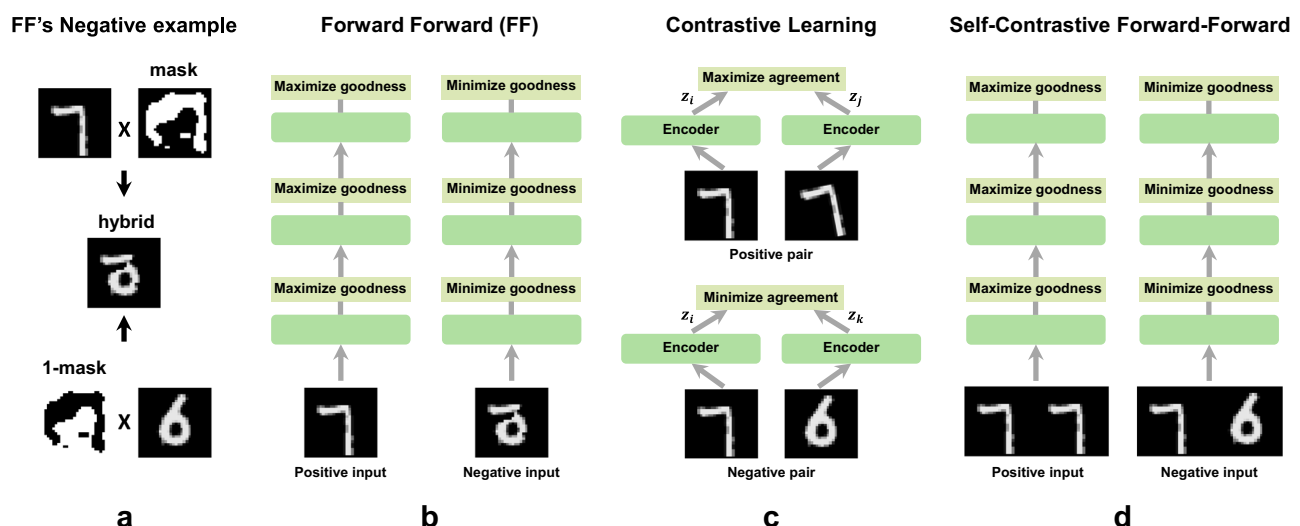


Fig. 1 | Comparative diagram illustrating three distinct unsupervised (self-supervised) learning paradigms. a Generation of a negative example is implemented by hybridization of two different images in the original FF paper²⁷. **b** In Forward Forward (FF) Learning, the layer-wise loss function is defined so as to maximize the goodness for positive inputs (real images) and minimize the goodness for negative inputs, each of which is generated by corrupting the real image to form a fake image, as shown in (a). **c** In Contrastive Learning, two encoders independently process input images. The

model is trained to maximize agreement between the latent representations of z_i and z_j , which are obtained from two augmented views of the same image, and to minimize agreement between the representations z_i and z_k , which are derived from different images. **d** Our proposed Contrastive Forward Forward Learning algorithm combines the principles of Forward Forward Learning and Contrastive Learning algorithms to maximize the goodness for concatenated similar pairs and minimize the goodness for dissimilar pairs with a layer-wise loss function.

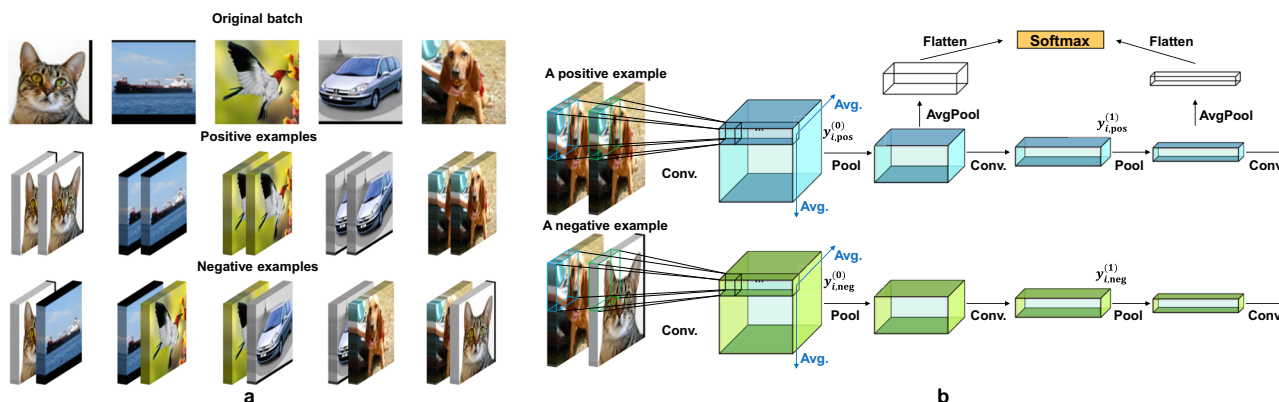


Fig. 2 | SCFF method for processing with convolutional neural network architecture. **a** The original batch of images (top row) is taken from the STL-10 dataset³⁰ and is used to generate positive (middle row) and negative examples (bottom row) for demonstration. **b** The generated positive and negative examples undergo a series of convolutional (Conv.) and pooling (AvgPool or Maxpool) operations to

extract relevant features. The blue axes labeled “Avg.” indicate that the goodness-based loss is computed across the channel dimension and then averaged along the height and width dimensions. Output neurons, extracted from each hidden layer via an external average pooling layer, are flattened and concatenated before being passed to a softmax layer for final classification.

their representations in feature space as done in contrastive self-supervised learning (Fig. 1c). More specifically, given a batch of N training examples, and for a randomly selected example $\mathbf{x}_k$ ($k \in \{1, N\}$) in the batch, the positive example $\mathbf{x}_{i,\text{pos}}^{(0)}$ (the number 0 is the layer index) is the concatenation of two repeated $\mathbf{x}_k$, i.e., $\mathbf{x}_{i,\text{pos}}^{(0)} = [\mathbf{x}_k, \mathbf{x}_k]^T$. The negative example $\mathbf{x}_{j,\text{neg}}^{(0)}$ is obtained by concatenating $\mathbf{x}_k$ with another example $\mathbf{x}_n$ ($n \neq k$) in the batch, i.e., $\mathbf{x}_{j,\text{neg}}^{(0)} = [\mathbf{x}_k, \mathbf{x}_n]^T$ (or $[\mathbf{x}_n, \mathbf{x}_k]^T$). Figure 2a shows some instances of generated positive and negative examples from the original training batch for the STL-10 dataset.

Importantly, although the input size is doubled, the computational cost and memory usage of network training are not impacted compared to standard FF because the weight matrices connecting $\mathbf{x}_k$ and $\mathbf{x}_n$ can be identical. For example, in the fully connected network illustrated in Fig. 1d, the outputs for the positive and negative examples from the first layer can be respectively written as:

$$\mathbf{y}_{i,\text{pos}}^{(0)} = f(W_1 \mathbf{x}_k + W_2 \mathbf{x}_k), \quad \mathbf{y}_{j,\text{neg}}^{(0)} = f(W_1 \mathbf{x}_k + W_2 \mathbf{x}_n), \quad (1)$$

where f is the ReLU activation function, and the weight matrices W_1 and W_2 correspond to the connections for each of the two images constituting the input. In practice, we set $W_1 = W_2$ because the gradient of the loss function with respect to W_1 and W_2 converges to the same value. This results in a reformulation where the inputs are summed rather than concatenated, yielding:

$$\mathbf{y}_{i,\text{pos}}^{(0)} = f(W(\mathbf{x}_k + \mathbf{x}_k)), \quad \mathbf{y}_{j,\text{neg}}^{(0)} = f(W(\mathbf{x}_k + \mathbf{x}_n)), \quad (2)$$

where $W = W_1 = W_2$. This reformulation ensures that the effective input size remains unchanged compared to a standard single-image input, and the feature map dimensions in subsequent layers are not affected by the initial pairing operation. Unlike naive concatenation, which would increase feature map width, our approach maintains computational efficiency while preserving the relational structure of the input pairs.

Intuitively, this can be understood by recognizing that swapping the positions of $\mathbf{x}_k$ and $\mathbf{x}_n$ in the input should not affect the output neural activities. A rigorous mathematical proof of the convergence of matrices W_1 and W_2 is provided in Supplementary Information A.

Training procedure

The training procedure builds upon the FF framework, enhanced by additional unsupervised training techniques:

- **Greedy Layer-wise Training/Joint Training:** SCFF supports two training paradigms: greedy layer-wise training and joint training, allowing for flexibility in optimization strategy based on computational constraints and architectural requirements. In greedy layer-wise training, each layer of the network is fully trained before proceeding to the next layer. In joint training, all layers are updated simultaneously using SCFF’s local learning rule in each iteration. After unsupervised training with SCFF, we froze the network and trained a linear downstream classifier^{16,52} with the backpropagation method on representations created by the network using the labeled data. The linear classifier was optimized using cross-entropy loss. The accuracy of this classification serves as a measure of the quality of the learned representations.
- **Goodness Function:** SCFF employs a ‘goodness’ function at each layer, similar to FF. The goodness score, $G_i^{(l)} = \frac{1}{M^{(l)}} \sum_m y_{i,m}^{2(l)}$, where l is the layer index and m represents the neuron index, calculated as the sum of squared activations, is optimized such that positive examples have higher goodness than negative examples.
- **Loss Function:** Predefined positive and negative examples are successively presented to the network’s input. The possibility of a positive example $\mathbf{x}_i$ being recognized as positive and a negative example $\mathbf{x}_j$ being recognized as negative by the network are defined as $p_{\text{pos}}(\mathbf{x}_i) = \sigma(G_i^{(l)} - \theta_{\text{pos}}^{(l)})$ and $p_{\text{neg}}(\mathbf{x}_j) = \sigma(\theta_{\text{neg}}^{(l)} - G_j^{(l)})$ respectively. The sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$ evaluates the effectiveness of the separation, where $\theta_{\text{pos}}^{(l)}$ and $\theta_{\text{neg}}^{(l)}$ are fixed values that serve as hyperparameters of the network.

The loss function encourages the network to increase the goodness for positive examples so that it significantly exceeds the threshold ($p_{\text{pos}}(\mathbf{x}_i) \rightarrow 1$) and to decrease the goodness score for negative input examples so that it falls well below the threshold ($p_{\text{neg}}(\mathbf{x}_j) \rightarrow 1$). At layer l , the loss function is defined as:

$$\begin{aligned} \mathcal{L}_{\text{FF}} &= -\mathbb{E}_{\mathbf{x}_i \sim \text{pos}} \log p_{\text{pos}}(\mathbf{x}_i) - \mathbb{E}_{\mathbf{x}_j \sim \text{neg}} \log p_{\text{neg}}(\mathbf{x}_j) \\ &= -\mathbb{E}_{G_{i,\text{pos}}^{(l)}} [\log \sigma(G_{i,\text{pos}}^{(l)} - \theta_{\text{pos}}^{(l)})] - \mathbb{E}_{G_{j,\text{neg}}^{(l)}} [\log \sigma(\theta_{\text{neg}}^{(l)} - G_{j,\text{neg}}^{(l)})] \end{aligned} \quad (3)$$

where $G_{i,\text{pos}}^{(l)}$ and $G_{j,\text{neg}}^{(l)}$ respectively correspond to the goodness for the positive and negative examples input at layer l . The final loss is computed over all N examples in the batch.

- **Normalization and Standardization**

To ensure stability during training, SCFF applies two key pre-processing steps:

- **Dataset-wide Normalization:** Each input image is normalized by subtracting the mean and dividing by the standard deviation, calculated per channel across the entire dataset.
- **Per-image Standardization:** Each individual image is standardized so that its pixel values have a mean of 0 and a standard deviation of 1. This ensures consistent input scaling, which is particularly important for unsupervised learning.

Other techniques include the “Triangle” method^{50,53} for transmitting information between layers, adding a penalty term in the loss function to ensure stable training, and applying an extra pooling layer to retrieve information at each layer. For further details, see the “Methods” section. All details about the impact of hyperparameters and the training of the linear classifier are provided in Supplementary Information G and H.

Experiments

We evaluate SCFF on different image datasets, including MNIST⁴⁷, CIFAR-10⁴⁸, Tiny ImageNet⁴⁹, and STL-10⁵⁰ (results in Table 1 and

Fig. 3), as well as an audio dataset FSDD⁵¹ (results in Fig. 4). Across all benchmarks, SCFF surpasses all state-of-the-art algorithms that combine a local learning rule with fully-forward operation, while maintaining its advantage in hardware adaptability.

Multilayer perceptron (MLP): MNIST. On the MNIST dataset, SCFF achieves a test accuracy of $98.70\% \pm 0.01\%$ when trained on a two-layer fully-connected network (MLP) with 2000 hidden neurons per layer, which is comparable to the performance achieved by backpropagation²⁷. This surpasses previously published benchmarks on other local learning algorithms applied to MLPs, including 97.8% in ref. 54, 98.42% in ref. 55 (supervised training), and 96.6% in ref. 56. The full comparisons are presented in Tables 2, 3, and 4.

Convolutional neural networks (CNN): MNIST, CIFAR-10, and Tiny ImageNet. The convolutional neural network (CNN) processes three-dimensional color images. The original images are concatenated along the channel dimension to form positive or negative inputs (see Fig. 2). The output of each convolutional layer is represented as a three-

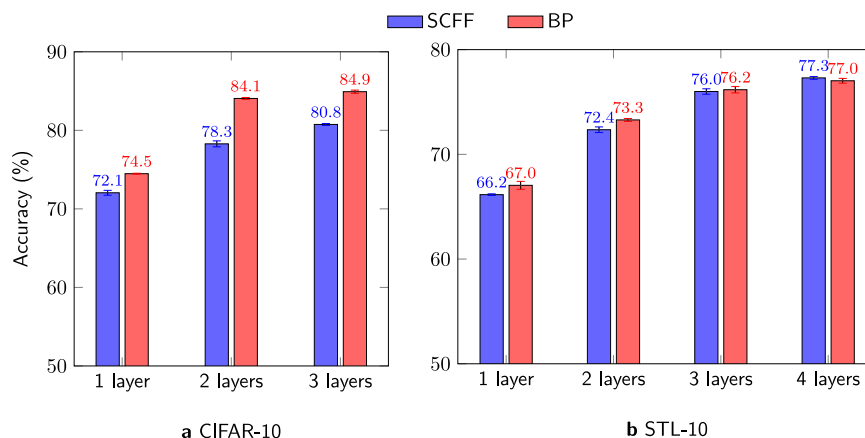


Fig. 3 | Comparison of test accuracy across different network depths using SCFF and backpropagation methods. Blue and red bars represent results obtained using SCFF and Backpropagation (BP), respectively, in (a) (CIFAR-10) and

(b) (STL-10). Error bars represent the mean $\pm$ standard deviation across three runs with different random seeds.

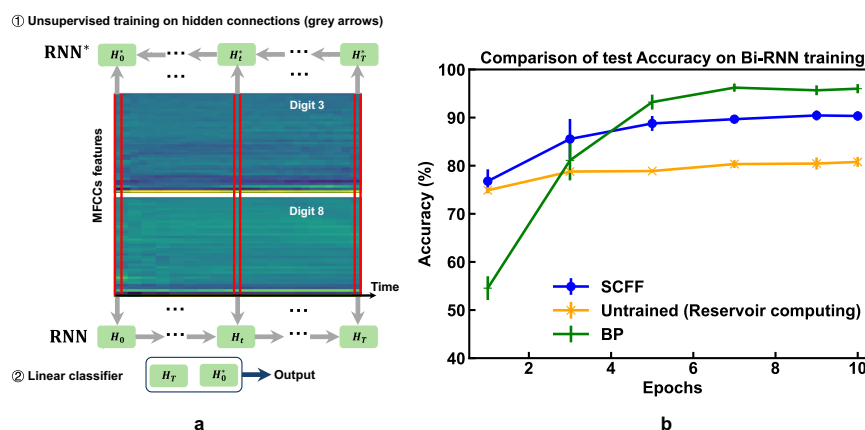


Fig. 4 | Bi-directional RNN (Bi-RNN) results on FSDD dataset. a Training procedure of SCFF on a Bi-RNN. In the first stage, unsupervised training is performed on the hidden connections (both input-to-hidden and hidden-to-hidden transformations) using positive and negative examples. Positive examples are created by concatenating two identical MFCC feature vectors of a digit along the feature dimension, while negative examples are generated by concatenating MFCCs from two different digits (e.g., digit 3 and digit 8), as illustrated in the figure. At each time step, the features are sequentially fed into the Bi-RNN (forward RNN and backward

RNN). The red regions indicate features at different time steps. In the second stage, a linear classifier is trained using the final hidden states from both RNNs, i.e., H_T and H'_0 as inputs for the classification task. **b** Comparison of test accuracy for the linear classifier trained on Bi-RNN outputs. The yellow curve represents accuracy with untrained (random) hidden neuron connections, the blue curve shows results from training with SCFF, the green curve shows Backpropagation (BP) results. Error bars represent the mean $\pm$ standard deviation across three runs with different random seeds.

Table 2 | Test accuracy [%] (top-1) comparison between SCFF and self-supervised Learning methods on MNIST, CIFAR-10, STL-10, and Tiny ImageNet datasets

Method	Local	Unsupervised	MNIST	CIFAR-10	STL-10	Tiny ImageNet
SimCLR (Chen et al. 2020 ⁷⁵)	x	✓	–	94.0	89.7	53.4 ⁹²
Bio-SSL (Tang et al. 2022 ⁷⁸)	✓	✓	–	72.7	68.8	–
PNN-SSL (Laydevant et al. 2023 ⁷⁷)	x	x	96.6	77.0	–	–
CLAPP (Illing et al. 2021 ⁷⁹)	✓	✓	–	–	73.6	–
SCFF (ours)	✓	✓	98.7	80.8	77.3	35.7

The symbols ✓, **x**, and – mean “yes”, “no”, and “no reported results”, respectively. Entries in bold indicate the proposed method and its associated results.

Table 3 | Test accuracy [%] comparison between SCFF and other forward-only methods with local learning rules on MNIST (top-1), CIFAR-10 (top-1), STL-10 (top-1) datasets, and Tiny ImageNet (top-5)

Method	Unsupervised	MNIST	CIFAR-10	STL-10	Tiny ImageNet
SigProp (Kohan et al. 2023 ⁴⁶)	x	98.2	91.6	–	–
PEPITA (Srinivasan et al. 2023 ⁵⁵)	x	98.4	53.8	–	–
Act. Learning (Zhou et al. 2022 ⁵⁶)	✓	97.1	58.7	–	–
HardHebb (Miconi et al. 2021 ⁵³)	✓	–	64.8	–	–
HardHebb (Lagani et al. 2021 ⁵⁸)	✓	98.5 ⁷⁹	65.9	–	37.0
Hebb-CHU (Krotov et al. 2019 ⁹³)	✓	98.5	50.8	–	–
Hebb-PNorm (Grinberg et al. 2019 ⁹⁴)	✓	–	72.2	–	–
SoftHebb (Journé et al. 2022 ¹⁶)	✓	97.8	80.3	76.2	–
SCFF (ours)	✓	98.7	80.8	77.3	59.8

The symbols ✓, **x**, and – mean “yes”, “no”, and “no reported results”, respectively. Entries in bold indicate the proposed method and its associated results.

Table 4 | Test accuracy [%] comparison between SCFF and energy-based methods with local learning rules on the MNIST, CIFAR-10, and STL-10 datasets

Method	Unsupervised	MNIST	CIFAR-10	STL-10
EqProp (Laborieux et al. 2021 ²⁴)	x	98.0	88.6	–
EqProp (Liu et al. 2024 ⁸²)	✓	97.6	71.5	–
DualProp (Højer et al. 2023 ²²)	x	98.4	92.3	–
SCFF (ours)	✓	98.7	80.8	77.3

The symbols ✓, **x**, and – mean “yes”, “no”, and “no reported results”, respectively. Entries in bold indicate the proposed method and its associated results.

dimensional vector $\mathbf{y}_{i,\text{pos}}^{(l)}$ (or $\mathbf{y}_{i,\text{neg}}^{(l)} \in \mathbb{R}^{C \times H \times W}$). The Loss function at layer l is then defined as:

$$\mathcal{L}_{\text{SCFF}} = -\mathbb{E}_{G_{i,\text{pos}}^{(l)}} \left[\frac{1}{H \times W} \sum_h \sum_w \log \sigma \left(G_{i,h,w,\text{pos}}^{(l)} - \theta_{\text{pos}}^{(l)} \right) \right] - \mathbb{E}_{G_{j,\text{neg}}^{(l)}} \left[\frac{1}{H \times W} \sum_h \sum_w \log \sigma \left(\theta_{\text{neg}}^{(l)} - G_{j,h,w,\text{neg}}^{(l)} \right) \right] \quad (4)$$

where the goodness of neural activities is calculated over the channels as $G_{i,h,w,\text{pos}}^{(l)} = \frac{1}{C} \sum_c y_{i,c,h,w,\text{pos}}^{2(l)}$ (or $G_{i,h,w,\text{neg}}^{(l)} = \frac{1}{C} \sum_c y_{i,c,h,w,\text{neg}}^{2(l)}$).

We evaluate SCFF using a three-layer CNN with the same architecture as the state-of-the-art SoftHebb, using 96, 384, and 1536 filters respectively¹⁶. On MNIST, our results show that SCFF achieves a test accuracy of $99.37\% \pm 0.06\%$ when trained with a CNN, comparable to the 99.35% achieved by SoftHebb¹⁶.

For the CIFAR-10 dataset, SCFF achieves an accuracy of $80.75\% \pm 0.12\%$, surpassing the previous state-of-the-art accuracies for

purely-forward unsupervised learning, of 80.3% on CIFAR-10 achieved using the SoftHebb algorithm¹⁶.

We also compared the test accuracies across different network depths using SCFF and backpropagation methods, as shown in Fig. 3a. The comparison reveals that SCFF can effectively capture complex representations at deeper layers, similar to BP. Additionally, the evolution of accuracy with more layers indicates the scalability of SCFF and its robustness in deeper architectures.

To further evaluate SCFF on more complex datasets, we apply an AlexNet-inspired architecture with five convolutional layers (filter sizes: 64-192-384-256-256) to Tiny ImageNet, which is a subset of ImageNet⁵⁷ containing 200 object classes. SCFF achieves a top-1 accuracy of $35.67\% \pm 0.42\%$ and a top-5 accuracy of $59.75\% \pm 0.18\%$, significantly outperforming previous attempts to apply local learning algorithms to this database. Specifically, SCFF surpasses the Hebbian learning-based approach of ref. 58, which achieved a top-5 accuracy of 36.99%, and the Inference Learning Algorithm, which achieved a top-5 accuracy of 47.53%⁵⁹. These results highlight SCFF’s ability to learn hierarchical representations efficiently in larger-scale image classification tasks.

STL-10: semi-supervised learning. The STL-10 dataset is designed for semi-supervised learning tasks, where the majority of the data is unlabeled. We train SCFF using a four-layer CNN with 96, 384, 1536, and 6144 filters, respectively¹⁶. We compare its performance against both traditional supervised learning methods and other local learning algorithms.

Notably, for STL-10, SCFF achieved a final layer performance of $77.30\% \pm 0.12\%$, higher than the one of BP: $77.02\% \pm 0.22\%$ (Fig. 3b). This is because the STL-10 dataset contains a large amount of unlabeled images, which limits the effectiveness of supervised BP training. By fine-tuning SCFF with end-to-end BP on the few labelled STL-10 examples, SCFF’s accuracy further improves to 80.13%. This demonstrates that SCFF is highly suitable for unsupervised pretraining

followed by supervised BP training, making it ideal for semi-supervised learning approaches.

Unlike other unsupervised learning methods, where the result is obtained solely from the final layer's output, SCFF integrates neuron information with the linear classifier from intermediate layers, leading to more comprehensive feature extraction²⁷. For CIFAR-10 (Fig. 3a), the test accuracy for the two-layer and three-layer models was obtained by combining the outputs of all previous layers (pooled information for dimensionality reduction; see "Methods" section) before feeding them into the final linear classifier. For the STL-10 dataset, we combined the outputs from both the third and fourth layers for the final classification, resulting in a 1% improvement in accuracy compared to using only the fourth layer's outputs as input to the linear classifier.

By visualizing and investigating the class activation map, which highlights the importance of each region of a feature map in relation to the model's output, we can intuitively observe that after four layers, more distinct and meaningful structures emerge (see Supplementary Information H). Specifically, the activation maps corresponding to higher-layer features focus on the general contours and key objects within the input, facilitating improved feature extraction and classification⁶⁰.

Comparison of greedy layer-wise training and joint training. We compared the results of greedy layer-wise training and joint training on multiple datasets. Our results indicate that both approaches yield comparable performance. For instance, on CIFAR-10, joint training reaches an accuracy of $80.60\% \pm 0.15\%$, whereas layer-wise training achieves $80.75\% \pm 0.12\%$. A similar trend is observed on STL-10, where joint training attains $77.14\% \pm 0.04\%$, while layer-wise training achieves $77.30\% \pm 0.14\%$. These findings indicate that layer-wise training does not degrade feature representations, as confirmed by stable accuracy trends across deeper layers.

The similarity in performance between these two training strategies is due to the fact that weight updates in earlier layers do not affect the normalized inputs passed to subsequent layers²⁷. Specifically, all activations in a given layer scale by the same factor after an update, and layer normalization cancels out this scaling effect. This ensures that greedy layer-wise training prevents error accumulation, maintains stability, and yields feature representations comparable to joint training. Full derivations can be found in Supplementary Information C.

Beyond accuracy, layer-wise training offers practical benefits: it enables more efficient hardware implementation and reduces memory constraints—making it particularly advantageous for neuromorphic computing. Additionally, training each layer independently allows for incremental learning and adaptability, making SCFF more flexible in scenarios where training resources are limited or where continual learning is desired.

Free Spoken Digit Dataset (FSDD): sequential data. Prior research in forward-only learning, including Fast Weights⁶¹, differentiable plasticity⁶², short-term plasticity^{63,64}, and e-prop¹⁸, has demonstrated the viability of forward-only approaches for sequence learning. While the original FF paper²⁷ includes a multi-layer recurrent neural network, it uses a static MNIST image repeated over time frames as input, with the objective of modeling top-down effects. Another implementation demonstrates a limited form of sequence learning with a fully connected network, but this architecture could not handle real-time sequential data due to the absence of recurrence. As a result, FF has yet to be extended to effectively handle recurrent network scenarios for time-varying inputs. One of SCFF's most significant advancements over FF is its ability to handle time-series data.

We employ the FSDD, a standard benchmark task for evaluating RNN training performance^{65–67}. The FSDD is a collection of audio recordings where speakers pronounce digits from 0 to 9 in English. We

follow the standard procedure that consist in extracting frequency domain information at different time intervals, here through Mel-Frequency Cepstral Coefficient (MFCC) features with 39 channels⁶⁸. Plots of the evolution of MFCC features with time are shown in Fig. 4 for the digits 3 and 8. The SCFF method forms positive and negative examples by concatenating the same input for positive examples, and different ones for negative examples. Figure 4a presents a negative example which is generated by concatenating MFCC features from two different digits. The goal of the task is to recognize the digit after feeding in the full sequence, from the output of the network at the last time step.

We train a Bi-directional Recurrent Neural Network (Bi-RNN) in an unsupervised way using the SCFF method to classify the digits. The procedure that we use for this purpose is illustrated in Fig. 4a. Unlike conventional uni-directional RNNs, where the sequential input is processed step by step in a single direction, resulting in a sequence of hidden states from H_0 to H_T (as depicted in the bottom RNN in Fig. 4a), the Bi-RNN comprises two RNNs that process the input in parallel in both forward and backward directions. This results in hidden states evolving from H_0 to H_T in the forward RNN and from H'_T to H'_0 in the backward RNN, as shown in the top portion of the figure. The red regions in the figure highlight the features at different time steps. This bidirectional structure is particularly advantageous for tasks where context from both preceding and succeeding audio frames is critical, such as speech recognition, enhancing model performance compared to conventional uni-directional RNNs⁶⁹.

The output of each directional RNN for a positive or negative input example is a two-dimensional vector $\mathbf{h}_i \in \mathbb{R}^{M \times T}$, where T represents the number of time steps and M denotes the number of hidden neurons. The loss function at layer l is then defined as:

$$\begin{aligned} \mathcal{L}_{\text{SCFF}} = & -\mathbb{E}_{G_{i,\text{pos}}^{(l)}} \left[\frac{1}{T} \sum_t \log \sigma(G_{i,t,\text{pos}}^{(l)} - \theta_{\text{pos}}^{(l)}) \right] \\ & -\mathbb{E}_{G_{j,\text{neg}}^{(l)}} \left[\frac{1}{T} \sum_t \log \sigma(G_{j,t,\text{neg}}^{(l)} - \theta_{\text{neg}}^{(l)}) \right] \end{aligned} \quad (5)$$

where the goodness of neural activities is calculated at each time step as $G_{i,t,\text{pos}}^{(l)} = \frac{1}{M} \sum_m h_{i,t,m,\text{pos}}^{2(l)}$ (or $G_{i,t,\text{neg}}^{(l)} = \frac{1}{M} \sum_m h_{i,t,m,\text{neg}}^{2(l)}$).

After the first stage of unsupervised training, a linear classifier is trained on the hidden states from the final time step in both directions, as shown in the bottom of the Fig. 4a. The blue, orange and green curves in Fig. 4b depict the test accuracy of the linear output classifier with hidden connections trained using SCFF, with random (untrained) hidden connections, and with backpropagation methods, respectively.

SCFF achieves a test accuracy of $90.33\% \pm 0.94\%$ when trained with a one-layer Bi-RNN containing 500 hidden neurons in each direction (refer to Supplementary Information F for further architectural details). It is below the performance of backPropagation Though Time that easily reaches $96.00\% \pm 0.94\%$ accuracy on this small task. However, SCFF avoids the issues of vanishing and exploding gradients of backpropagation through time (BPTT), as the gradients at each time step are calculated independently. This eliminates the dependency between time steps, providing a more stable training process which could be useful for future experiments on larger networks.

Furthermore, the SCFF results are well above the model with untrained (random) input and hidden connections which plateaus at $80.78\% \pm 1.03\%$. Such model, in which only the output connections are trained, is akin to Reservoir Computing, a method that is often used to leverage physical systems on sequential data for neuromorphic applications⁷⁰. SCFF provides a way to train these input and hidden layer connections in a simple, hardware-compatible way, and opens the path to a considerable gain of accuracy. This achievement opens the door for its extension to more complex tasks involving temporal

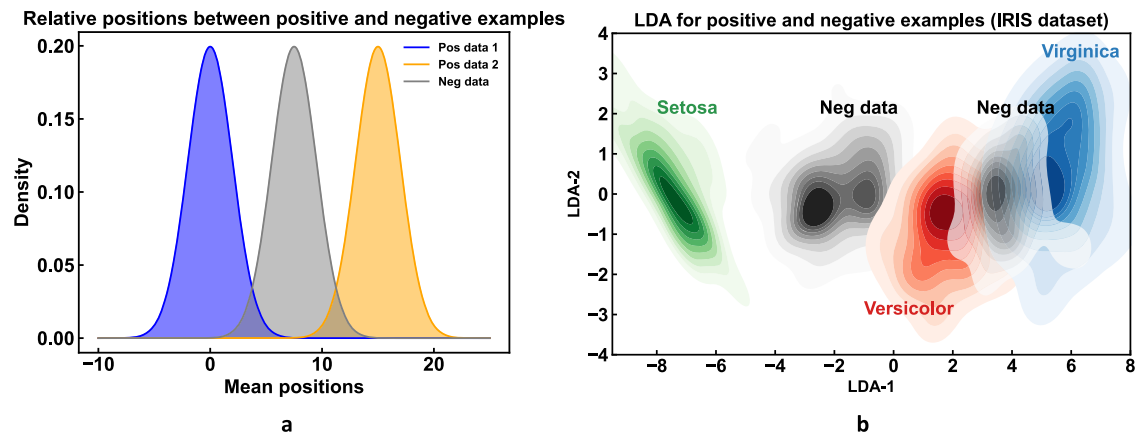


Fig. 5 | Probability distributions of relative positions between positive and negative examples. a Theoretical distributions of positive examples from two different classes (denoted as Pos data 1 and Pos data 2) with distinct means ($2\mu_1 = 0$ and $2\mu_2 = 15$) and identical variance ($2\Sigma = 4$) are shown with blue and orange curves, respectively. The theoretical distribution of negative examples (denoted as Neg

data) derived from the two classes using the formula (7) is depicted by the grey curve. **b** Continuous probability density of linear discriminant analysis (LDA) applied to the IRIS dataset, displaying contours for positive examples in green (Setosa), red (Versicolor), and blue (Virginica), and for negative examples (Neg Data) in grey.

sequences and its potential use in neuromorphic computing domains, such as dynamic vision sensors⁷¹.

Overall, this result constitutes the first successful application of the FF approach to sequential data in an unsupervised manner. Future work could explore feedback mechanisms to enable top-down contextual modulation or extending SCFF with memory-augmented mechanisms, similar to Fast Weights⁶¹, to improve temporal information retention.

Discussion

Analysis of the negative examples

The improvements brought by SCFF rely on an approach for consistently constructing positive and negative examples, in a way that can be applied to any database.

The effectiveness of the negative examples in SCFF can be understood through the lens of Noise Contrastive Estimation (NCE). In NCE, a key insight is that “the noise distribution should be close to the data distribution, because otherwise, the classification problem might be too easy and would not require the system to learn much about the structure of the data”⁷². Our method of generating the positive and negative examples aligns with this principle if we treat the negative examples as “noise data”. We assume that the data samples for each class follow a multivariate Gaussian distribution with a shared covariance matrix Σ and that each class is statistically independent of the others—assumptions commonly employed in various statistical models⁷³.

Since the input weight matrices are identical, i.e., $W = W_1 = W_2$, the input transformation simplifies to: $\mathbf{y}_{i,\text{pos}}^{(0)} = f(W(\mathbf{x}_k + \mathbf{x}_k)) = f(W(2\mathbf{x}_k))$, $\mathbf{y}_{j,\text{neg}}^{(0)} = f(W(\mathbf{x}_k + \mathbf{x}_n))$. Thus, the positive and negative examples can be interpreted as:

$$\mathbf{x}_{i,\text{pos}}^{(0)} = 2\mathbf{x}_k, \quad \mathbf{x}_{j,\text{neg}}^{(0)} = \mathbf{x}_k + \mathbf{x}_n. \quad (6)$$

Therefore, the distributions of positive examples $\mathbf{x}_{i,\text{pos}}^{(0)}$ and negative examples $\mathbf{x}_{j,\text{neg}}^{(0)}$ follow:

$$\begin{aligned} \mathbf{x}_{i,\text{pos}}^{(0)} &\sim \mathcal{N}(2\boldsymbol{\mu}_1, 2\Sigma) \\ \mathbf{x}_{j,\text{neg}}^{(0)} &\sim \mathcal{N}(\boldsymbol{\mu}_1 + \boldsymbol{\mu}_2, 2\Sigma) \end{aligned} \quad (7)$$

where $\boldsymbol{\mu}_1$ and $\boldsymbol{\mu}_2$ are means of two different classes, respectively.

Theoretically, the negative examples always lie somewhere between two different clusters of positive examples in the sample

space, as illustrated in Fig. 5a for the one-dimensional case. Unlike conventional contrastive learning methods that require explicit hard negative sampling, SCFF inherently constructs negative examples that are statistically distinct yet sufficiently similar to the data distribution. This ensures that SCFF maintains a strong contrastive signal without relying on additional augmentation strategies. For practical analysis with a real-world dataset, we visualized the distributions of positive and negative examples from the IRIS dataset⁷⁴ using 2D linear discriminant analysis (LDA), which distinguishes between three different types of irises, as shown in Fig. 5b. This visualization shows that the negative examples are positioned between different clusters of positive examples, suggesting that they contribute to pushing apart and further separating adjacent positive examples as they are mapped into higher-dimensional space during training. Additionally, negative examples are formed by combining two examples from different classes, enriching the diversity of negative examples and leading to more robust training. For a detailed analysis of how the LDA components evolve during training as the input data is mapped into the feature space and more theoretical results, please refer to Supplementary Information B.

Comparison to the original FF algorithm

In SCFF, we have expanded the applicability of FF to complex unsupervised tasks beyond the MNIST dataset. The SCFF method achieves state-of-the-art (SOTA) accuracy for local methods on challenging datasets such as CIFAR-10, Tiny ImageNet and STL-10, largely outperforming the original FF algorithm (see Table 1). This is a significant advancement, as it demonstrates that the method performs comparably to other local and forward-only algorithms in complex visual tasks, thereby broadening the scope and utility of FF to unlabeled data processing.

We have also shown that FF can solve sequential tasks. This extension is crucial for applications in time-series analysis and other domains where data is inherently sequential. By incorporating these improvements, our SCFF method not only overcomes the original limitations of FF but also sets a new benchmark for unsupervised learning algorithms in terms of versatility and performance.

Comparison to SOTA self-supervised learning (SSL)

The SCFF method is inspired from self-supervised contrastive learning techniques^{75,76}. While its purely local, layer-wise learning may limit its accuracy compared to end-to-end trained SSL models (Table 2), this design offers unique advantages. SCFF indeed operates without the

Table 5 | Single-layer training performance (in terms of epoch duration) for different publicly available forward-only and local learning repositories, compared to SCFF, on CIFAR-10

Method	Unsupervised	Epoch duration (s)	Acc. (%)
SigProp (Kohan et al. 2023 ⁴⁶)	x	7.5	91.6
HardHebb (Miconi et al. 2021 ⁵³)	✓	1.6	64.8
HardHebb (Lagani et al. 2022 ⁷⁹)	✓	5.1	64.6
SoftHebb (Journé et al. 2022 ¹⁶)	✓	2.6	80.3
SCFF (ours)	✓	1.2	80.8

Accuracy results of the overall models from the respective papers on CIFAR-10 are also reported. All experiments were conducted on a single NVIDIA RTX 4090 GPU. Entries in bold indicate the proposed method and its associated results.

auxiliary heads (multi-layer nonlinear projector) nor the complex regularization techniques required in global SSL methods, which simplifies its implementation and makes it more suitable for neuromorphic computing applications. SCFF's local loss and layer normalization also ensure stable activations and mitigate the vanishing gradient problems that typically arise in deep networks trained via backpropagation.

Another key distinction between SCFF and standard contrastive learning approaches lies in the way how negative samples are handled. Traditional methods such as SimCLR⁷⁵ and MoCo⁷⁶ require large batches or memory banks to construct multiple negative pairs, often incorporating hard negatives and strong augmentations to enhance contrastive separation. These operations are computationally expensive and time-consuming. In contrast, SCFF maintains a fixed 1:1 ratio of positive to negative samples by optimizing a goodness-based objective, eliminating the need for additional transformations or augmentations. This formulation ensures that SCFF remains computationally efficient while still preserving a meaningful contrastive signal. Future work could explore enhancements such as integrating top-down feedback connections and incorporating feedback-informed negative selection, where model predictions dynamically refine the selection of negative samples to enhance contrastive separation to further scale SCFF to modern deep networks like ResNet-50 and Vision Transformers.

Recent developments in local versions of contrastive self-supervised learning have shown promising results^{77,78}. For instance, Laydevant et al.⁷⁷ empirically demonstrated that layer-wise SSL objectives can be optimized rather than a single global one, achieving performance comparable to global optimization on datasets such as MNIST and CIFAR-10 (see Table 2). However, this accuracy comes from multi-layer MLPs as projector heads at each layer, increasing the computational complexity. Illing et al.¹⁷ have shown that local plasticity rules, when applied through the CLAPP model, can successfully build deep hierarchical representations without the need for backpropagation. However, this method introduces additional processing along the time axis, which may add complexity when dealing with data that lacks temporal dynamics.

Comparison to other forward-only methods

The development of purely forward learning techniques has been historically driven by their potential for biologically plausible and neuromorphic computing applications^{55,56}. We compare recent and SOTA results of purely-forward methods with local learning rules to SCFF in Table 3.

Similar to FF, Pepita⁵⁵ processes data samples in two forward passes. The first pass is identical to FF, while the input of the second pass is modulated by incorporating information about the error from the first forward pass through top-down feedback. Activation Learning⁵⁶ builds on Hebb's well-known proposal, discovering unsupervised features through local competitions among neurons. However, these methods do not yet scale to more complex tasks, limiting their potential applications.

Hebbian deep learning has also achieved remarkable progress recently^{16,53,79,80}. These methods are purely local in space and can be applied purely locally in time, offering a biologically plausible approach to learning. Miconi⁵³ demonstrated that Hebbian learning in hierarchical convolutional neural networks can be implemented with modern deep learning frameworks by using specific losses whose gradients produce the desired Hebbian updates. However, adding layers has not resulted in significant performance improvements on standard benchmarks^{53,79}. Journé et al.¹⁶ proposed using a simple softmax to implement a soft Winner-Takes-All and derived a Hebbian-like plasticity rule (SoftHebb). With techniques like triangle activation and adjustable rectified polynomial units, SoftHebb achieves increased efficiency and biological compatibility, enhancing accuracy compared to state-of-the-art biologically plausible learning methods.

Our SCFF method brings the FF approach to accuracy levels comparable to SoftHebb, effectively bridging the gap between these learning paradigms. A key advantage of Hebbian learning is its ability to learn without contrast, much like non-contrastive self-supervised learning techniques, operating purely in an unsupervised manner. Conversely, FF is flexible regarding labels, akin to contrastive self-supervised learning techniques, supporting both unsupervised learning as we demonstrate here with SCFF and supervised learning. This versatility allows FF to be applied across a broader range of tasks and datasets, enhancing its applicability and effectiveness in diverse scenarios.

Another notable distinction between SCFF and SoftHebb lies in the way the classifier is applied to the neural network. SoftHebb attaches a linear classifier to the final layer, utilizing all neurons from the last layer as inputs. In contrast, SCFF applies an additional pooling operation to aggregate information from intermediate layers, reducing the total number of input neurons before feeding them into the linear classifier. This results in a more compact and efficient representation. For example, on CIFAR-10, SoftHebb employs 24,576 input neurons for classification, whereas SCFF only uses 18,432 due to the added pooling operation. Similarly, for STL-10, SoftHebb utilizes 221,184 input neurons, while SCFF significantly reduces this number to 38,400. This reduction in the number of neurons helps simplify the classification process and contributes to computational efficiency.

Additionally, our method provides a computational advantage. Table 5 presents a comparison of single-layer training time per epoch on the CIFAR-10 dataset. To ensure a fair comparison, all methods are evaluated under a common setting: a single convolutional layer with 5×5 filters, 3 input channels, and 96 output channels, using their respective training methodologies. The results show that SCFF enables more efficient training of convolutional layers compared to prior methods. This efficiency stems from SCFF's learning rule, which eliminates weight transpositions and matrix multiplications during gradient computation, thereby reducing computational complexity. In contrast, algorithms such as SoftHebb rely on these operations, leading to increased training time.

Comparison to energy-based learning methods

Energy-based learning methods (Table 4), such as Equilibrium Propagation (EP), Dual Propagation, and Latent Equilibrium^{20,22,81}, also offer

locality in space and time. These methods have a significant advantage over SCFF due to their strong mathematical foundations, closely approximating gradients from BP and BPTT. This theoretical rigor allows them to be applied to a wide range of physical systems, making them particularly appealing for neuromorphic computing applications. EP, for instance, can operate in an unsupervised manner⁸², while recent advancements in Generalized Latent Equilibrium⁸³ have extended these models to handle sequential data effectively.

However, the implementation of energy-based methods poses certain challenges. Specifically, the backward pass in these methods requires either bidirectional neural networks or dedicated backward circuits^{84,85}. These requirements can be complex to design and build in a manner that is both energy-efficient and compact. In contrast, the simplicity and versatility of SCFF in supporting both supervised and unsupervised learning, without the need for complex backward circuitry, make it a practical alternative for many applications⁸⁶. This balance of accuracy, ease of implementation, and versatility underscores the potential of SCFF in advancing neuromorphic computing systems.

In conclusion, the FF algorithm has sparked significant advancements in both biologically-inspired deep learning and hardware-efficient computation. However, its original form faced challenges in handling complex datasets and time-varying sequential data. Our method, Self Contrastive Forward-Forward (SCFF), addresses these limitations by integrating contrastive self-supervised learning principles directly at the input level, enabling the generation of positive and negative examples through a simple concatenation of input data. SCFF not only surpasses existing unsupervised learning algorithms in accuracy on datasets like MNIST, CIFAR-10, and STL-10 but also successfully extends the FF approach to sequential data, demonstrating its applicability to a broader range of tasks. These developments pave the way for more robust and versatile applications of FF in both neuromorphic computing and beyond, opening new avenues for research and practical implementations in the field.

Methods

SCFF learns representations by maximizing agreement (increasing activations/goodness) between concatenated pairs of identical data examples while minimizing agreement (reducing activations/goodness) between concatenated pairs of different data examples using a cross-entropy-like loss function at each layer. The network is trained layer by layer, with each layer's weights being frozen before moving on to the next. Unlike the original FF framework, this approach incorporates several key components that contribute to achieving high accuracy across various tasks.

Normalization and standardization

For vision tasks, the data is typically normalized by subtracting the mean and dividing by the standard deviation for each channel. These mean and standard deviation values are computed across the entire training dataset, separately for each of the three color channels. This dataset-wide normalization centers the data, ensuring that each channel has a mean of 0 and is on a comparable scale.

In addition to dataset-wide normalization, we also applied per-image standardization, which plays an important role in unsupervised feature learning⁸⁷. Standardizing the images involves scaling the pixel values of each image such that the resultant pixel values of the image have a mean of 0 and a standard deviation of 1. This is done before each layer during processing^{50,53}, ensuring that each sample is centered, which improves learning stability and helps the network handle varying illumination or contrast between images.

Concatenation

The positive and negative examples (e.g., $\mathbf{x}_{i,\text{pos}}^{(0)}$ and $\mathbf{x}_{j,\text{neg}}^{(0)}$) are generated by concatenating two identical images for the positive examples

and two different images for the negative examples. After being processed by the first layer, the output vectors $\mathbf{y}_{i,\text{pos}}^{(0)}$ and $\mathbf{y}_{j,\text{neg}}^{(0)}$ are obtained. There are two approaches for generating the inputs to the next layer. The first approach is to directly use the first layer's output of the positive example $\mathbf{y}_{i,\text{pos}}^{(0)}$ as the positive input $\mathbf{x}_{i,\text{pos}}^{(1)}$ and the first layer output of the negative example $\mathbf{y}_{j,\text{neg}}^{(0)}$ as the negative input $\mathbf{x}_{j,\text{neg}}^{(1)}$ for the next layer (refer to the highlighted blue section in Algorithm 1 in Supplementary Information D). The second approach involves re-concatenating to form new positive and negative inputs for the next layer. This is done by treating the first layer's positive outputs as a new dataset and recreating the corresponding positive and negative examples, similar to how the original dataset was processed to generate the initial positive and negative examples (refer to the highlighted blue section in Algorithm 2 in Supplementary Information D).

Supplementary Information D details the workflows of Algorithm 1 and Algorithm 2, focusing on their different approaches to generating positive and negative examples after the first layer. In practice, Algorithm 1 tends to be more effective for training the lower layers immediately following the first layer, while Algorithm 2 shows better performance in training deeper layers. Specifically, for the CIFAR-10 dataset, Algorithm 1 is utilized to train the second layer, while Algorithm 2 is applied to train the third layer. Similarly, for the STL-10 dataset, Algorithm 1 is employed for training the second and third layers, and Algorithm 2 is used for the fourth layer.

Triangle method of transmitting the information

"Triangle" method was firstly introduced by Coates et al.⁵⁰ to compute the activations of the learned features by K-means clustering. This method was later found to be effective in other Hebbian-based algorithms^{16,53} for transmitting the information from one layer to the next. The method involves subtracting the mean activation (computed across all channels at a given position) from each channel, and then rectifying any negative values to zero before the pooling layer. This approach to feature mapping can be viewed as a simple form of "competition" between features while also promoting sparsity.

Importantly, the "Triangle" activation only determines the responses passed to the next layer; it does not influence the plasticity. The output used for plasticity at each position is given by $\mathbf{y}_{i,\text{pos}}^{(l)} = f^{(l)}(\mathbf{x}_{i,\text{pos}})$ and $\mathbf{y}_{j,\text{neg}}^{(l)} = f^{(l)}(\mathbf{x}_{j,\text{neg}})$, where $f^{(l)}$ refers to the convolutional operations followed by ReLU activation at layer l .

Penalty term

Training with the FF loss can lead to excessively high output activations for positive examples, which significantly drives positive gradients and encourages unchecked growth in their activation. To mitigate this, we introduce a small penalty term—the Frobenius Norm of the Goodness vector—into the training loss function. For outputs from a CNN layer, the goodness vector $G_{i,h,w,\text{pos}}^{(l)}$ is a two-dimensional matrix where each element represents the goodness calculated over the channel outputs processed by all filters under the same receptive field. In the case of Bi-RNN outputs, the goodness vector $G_{i,t,\text{pos}}^{(l)}$ is a one-dimensional matrix, with each element representing the goodness at each time step. When a large goodness value is computed for a positive example, it generates a negative gradient that reduces the activation, thereby preventing excessive growth. The impact of this penalty term on training performance is further analyzed in Supplementary Information G.

Additional pooling operation to retrieve the features

To assess the performance of the intermediate layers in image classification tasks, we apply an additional pooling operation (average or max pooling) to the output of the pooling layer. This reduces the dimensionality of the features and helps in selecting relevant neuron activities. This approach is inspired by the "four quadrant" method used in previous work^{50,53}, where local regions extracted from the

convolutional layer are divided into four equal-sized quadrants, and the sum of neuron activations in each quadrant is computed for downstream linear classification tasks.

Supplementary Information E provides detailed information on the specific architecture of this additional pooling layer for various tasks.

Training setup

All experiments were conducted on a server equipped with an NVIDIA GeForce RTX 4090 GPU (24 GB memory). Training and evaluation were implemented using PyTorch and executed on a single GPU.

Data availability

The datasets used in this study—IRIS⁷⁴, MNIST⁴⁷, CIFAR-10⁴⁸, STL-10⁵⁰, Tiny ImageNet⁴⁹, and FSDD (Free Spoken Digit Dataset)⁵¹—are publicly available online from the sources cited. All datasets are openly accessible and were obtained from established public repositories to support reproducibility and verification of the results.

Code availability

The code to reproduce the results is available at: <https://github.com/neurophysics-cnrs/contrastive-forward-forward>⁸⁸.

References

- Nahavandi, D., Alizadehsani, R., Khosravi, A. & Acharya, U. R. Application of artificial intelligence in wearable devices: opportunities and challenges. *Comput. Methods Prog. Biomed.* **213**, 106541 (2022).
- Cardinale, M. & Varley, M. C. Wearable training-monitoring technology: applications, challenges, and opportunities. *Int. J. Sports Physiol. Perform.* **12**, 2–55 (2017).
- Shi, W., Cao, J., Zhang, Q., Li, Y. & Xu, L. Edge computing: vision and challenges. *IEEE Internet Things J.* **3**, 637–646 (2016).
- Khouas, A. R., Bouadjenek, M. R., Hacid, H. & Aryal, S. Training machine learning models at the edge: a survey. Preprint at <https://arxiv.org/abs/2403.02619> (2024).
- Richards, B. A. et al. A deep learning framework for neuroscience. *Nat. Neurosci.* **22**, 1761–1770 (2019).
- Wang, W., Li, Y. & Wang, M. Difficulties and approaches in enabling learning-in-memory using crossbar arrays of memristors. *Neuromorphic Comput. Eng.* **4**, 032002 (2024).
- Marković, D., Mizrahi, A., Querlioz, D. & Grollier, J. Physics for neuromorphic computing. *Nat. Rev. Phys.* **2**, 499–510 (2020).
- Nøkland, A. Direct feedback alignment provides learning in deep neural networks. *Adv. Neural Inf. Process. Syst.* **29**, 1037–1045 (2016).
- Frenkel, C., Lefebvre, M. & Bol, D. Learning without feedback: fixed random learning signals allow for feedforward training of deep neural networks. *Front. Neurosci.* **15**, 629892 (2021).
- Wang, Z. et al. Optical training of large-scale transformers and deep neural networks with direct feedback alignment. Preprint at <https://arxiv.org/abs/2409.12965> (2024).
- Launay, J. et al. Hardware beyond backpropagation: a photonic co-processor for direct feedback alignment. Presented at the *Beyond Backpropagation Workshop*, NeurIPS 2020. Preprint at <https://arxiv.org/abs/2012.06373> (2020).
- Filipovich, M. J. et al. Silicon photonic architecture for training deep neural networks with direct feedback alignment. *Optica* **9**, 1323–1332 (2022).
- Neftci, E. O., Augustine, C., Paul, S. & Detorakis, G. Event-driven random back-propagation: enabling neuromorphic deep learning machines. *Front. Neurosci.* **11**, 324 (2017).
- Launay, J., Poli, I., Boniface, F. & Krzakala, F. Direct feedback alignment scales to modern deep learning tasks and architectures. *Adv. Neural Inf. Process. Syst.* **33**, 9346–9360 (2020).
- Hebb, D. O. *The Organization of Behavior: A Neuropsychological Theory*. Reprint edn. (Psychology Press, New York, NY, 2005).
- Journé, A., Rodriguez, H. G., Guo, Q. & Moraitis, T. Hebbian deep learning without feedback. In *The Eleventh International Conference on Learning Representations (ICLR)* <https://openreview.net/forum?id=8gd4M-Rj1> (OpenReview.net 2023).
- Illing, B., Ventura, J., Bellec, G. & Gerstner, W. Local plasticity rules can learn deep representations using self-supervised contrastive predictions. *Adv. Neural Inf. Process. Syst.* **34**, 30365–30379 (2021).
- Bellec, G. et al. A solution to the learning dilemma for recurrent networks of spiking neurons. *Nat. Commun.* **11**, 3625 (2020).
- Halvagal, M. S. & Zenke, F. The combination of Hebbian and predictive plasticity learns invariant object representations in deep sensory networks. *Nat. Neurosci.* **26**, 1906–1915 (2023).
- Scellier, B. & Bengio, Y. Equilibrium propagation: bridging the gap between energy-based models and backpropagation. *Front. Comput. Neurosci.* **11**, 24 (2017).
- Stern, M., Hexner, D., Rocks, J. W. & Liu, A. J. Supervised learning in physical networks: from machine learning to learning machines. *Phys. Rev. X* **11**, 021045 (2021).
- Høier, R., Staudt, D. & Zach, C. Dual propagation: accelerating contrastive Hebbian learning with dyadic neurons. In *International Conference on Machine Learning* (eds. Krause, A. et al.) 13141–13156 (PMLR, 2023).
- Scellier, B., Ernoult, M., Kendall, J. & Kumar, S. Energy-based learning algorithms for analog computing: a comparative study. *Adv. Neural Inf. Process. Syst.* **36**, 52705–52731 (2023).
- Laborieux, A. et al. Scaling equilibrium propagation to deep convnets by drastically reducing its gradient estimator bias. *Front. Neurosci.* **15**, 633674 (2021).
- Dillavou, S., Stern, M., Liu, A. J. & Durian, D. J. Demonstration of decentralized physics-driven learning. *Phys. Rev. Appl.* **18**, 014040 (2022).
- Laydevant, J., Marković, D. & Grollier, J. Training an Ising machine with equilibrium propagation. *Nat. Commun.* **15**, 3671 (2024).
- Hinton, G. The forward-forward algorithm: some preliminary investigations. Preprint at <https://arxiv.org/abs/2212.13345> (2022).
- Ororbia, A. & Mali, A. A. The predictive forward-forward algorithm. In *Proceedings of the Annual Meeting of the Cognitive Science Society*, Vol. 45, https://escholarship.org/uc/item/2bt596r3#article_main (Cognitive Science Society, 2023).
- Giampaolo, F., Izzo, S., Prezioso, E. & Piccialli, F. Investigating random variations of the forward-forward algorithm for training neural networks. In *2023 International Joint Conference on Neural Networks (IJCNN)* 1–7 <https://ieeexplore.ieee.org/document/10191727> (IEEE, 2023).
- Zhao, G. et al. The cascaded forward algorithm for neural network training. *Pattern Recognition* **161**, 111292 (2025).
- Lorberbom, G., Gat, I., Adi, Y., Schwing, A. & Hazan, T. Layer collaboration in the forward-forward algorithm. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38, 14141–14148 <https://ojs.aaai.org/index.php/AAAI/article/view/29324> (AAAI Press, 2024).
- Tosato, N. et al. Emergent representations in networks trained with the forward-forward algorithm. <https://openreview.net/forum?id=X6Ruh6qCFV> (2023).
- Lee, H.-C. & Song, J. Symba: symmetric backpropagation-free contrastive learning with forward-forward algorithm for optimizing convergence. Preprint at <https://arxiv.org/abs/2303.08418> (2023).
- Papachristodoulou, A., Kyrkou, C., Timotheou, S. & Theodoridis, T. Convolutional channel-wise competitive learning for the forward-forward algorithm. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38, 14536–14544 <https://ojs.aaai.org/index.php/AAAI/article/view/29369> (AAAI Press, 2024).

35. Reyes-Angulo, A. A. & Paheding, S. Forward-forward algorithm for hyperspectral image classification. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 3153–3161 <https://www.computer.org/csdl/proceedings-article/cvprw/2024/654700d153/20A2iOQTDYk> (IEEE Computer Society, 2024).
36. Scodellaro, R., Kulkarni, A., Alves, F. & Schröter, M. Training convolutional neural networks with the forward-forward algorithm. Preprint at <https://arxiv.org/abs/2312.14924> (2023).
37. Yang, Y. A theory for the sparsity emerged in the forward forward algorithm. Preprint at <https://arxiv.org/abs/2311.05667> (2023).
38. Gananath, R. Improved forward-forward contrastive learning. Preprint at <https://arxiv.org/abs/2405.03432> (2024).
39. Hwang, T., Seo, H. & Jung, S. Employing layerwise unsupervised learning to lessen data and loss requirements in forward-forward algorithms. Preprint at <https://arxiv.org/abs/2404.14664> (2024).
40. Kam, F.d. *Memory Consolidation by Deep-q Forward-Forward Learning in Games*. PhD thesis (University of Groningen, Faculty of Science and Engineering, 2024).
41. De Vita, F. et al. μ -ff: on-device forward-forward training algorithm for microcontrollers. In *2023 IEEE International Conference on Smart Computing (SMARTCOMP)*, 49–56 <https://ieeexplore.ieee.org/document/10207585> (IEEE, 2023).
42. Park, S.-H. et al. On-chip learning in vertical NAND flash memory using forward-forward algorithm. *IEEE Trans. Electron Devices* **71**, 3640–3644 (2024).
43. Momeni, A., Rahmani, B., Malléjac, M., Del Hougne, P. & Fleury, R. Backpropagation-free training of deep physical neural networks. *Science* **382**, 1297–1303 (2023).
44. Oguz, I. et al. Forward-forward training of an optical neural network. *Opt. Lett.* **48**, 5249–5252 (2023).
45. Wu, Y. et al. Distance-forward learning: enhancing the forward-forward algorithm towards high-performance on-chip learning. Preprint at <https://arxiv.org/abs/2408.14925> (2024).
46. Kohan, A., Rietman, E. A. & Siegelmann, H.T. Signal propagation: the framework for learning and inference in a forward pass. *IEEE Trans. Neural Netw. Learn. Syst.* **35**, 8585–8596 (2024).
47. Deng, L. The MNIST database of handwritten digit images for machine learning research. *IEEE Signal Process. Mag.* **29**, 141–142 (2012).
48. Krizhevsky, A., Nair, V. & Hinton, G. *CIFAR-10: Canadian Institute for Advanced Research*. <http://www.cs.toronto.edu/~kriz/cifar.html> (2010).
49. Le, Y. & Yang, X. Tiny imagenet visual recognition challenge. *CS 231N* **7**, 3 (2015).
50. Coates, A., Ng, A. & Lee, H. An analysis of single-layer networks in unsupervised feature learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, (eds. Gordon G., Dunson D., Dudik M.) 215–223 (JMLR Workshop and Conference Proceedings, 2011).
51. Jackson, Z. et al. Jakobovski/free-spoken-digit-dataset: v1. 0.8. Zenodo (2018).
52. Alain, G. Understanding intermediate layers using linear classifier probes. Presented at the *ICLR 2017 Workshop Track* <https://openreview.net/forum?id=HJ4-rAVtI> (2017).
53. Miconi, T. Hebbian learning with gradients: Hebbian convolutional neural networks with modern deep learning frameworks. Preprint at <https://arxiv.org/abs/2107.01729> (2021).
54. Moraitis, T., Toichkin, D., Journé, A., Chua, Y. & Guo, Q. Softhebb: Bayesian inference in unsupervised Hebbian soft winner-take-all networks. *Neuromorphic Comput. Eng.* **2**, 044017 (2022).
55. Srinivasan, R. F. et al. Forward learning with top-down feedback: empirical and analytical characterization. In *The Twelfth International Conference on Learning Representations (ICLR)* <https://openreview.net/forum?id=My7lkRNnL9> (Open-Review.net 2024).
56. Zhou, H. Activation learning by local competitions. Preprint at <https://arxiv.org/abs/2209.13400> (2022).
57. Deng, J. et al. Imagenet: a large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 248–255 <https://ieeexplore.ieee.org/document/5206848> (IEEE, 2009).
58. Lagani, G., Falchi, F., Gennaro, C. & Amato, G. Hebbian semi-supervised learning in a sample efficiency setting. *Neural Netw.* **143**, 719–731 (2021).
59. Alonso, N., Krichmar, J. & Neftci, E. Understanding and improving optimization in predictive coding networks. In *Proceedings of the AAAI Conference on Artificial Intelligence* Vol. 38, 10812–10820 <https://ojs.aaai.org/index.php/AAAI/article/view/28954> (AAAI Press, 2024).
60. Lafabregue, B., Weber, J., Gançarski, P. & Forestier, G. Grad centroid activation mapping for convolutional neural networks. In *2021 IEEE 33rd International Conference on Tools with Artificial Intelligence (ICTAI)*, 184–191 <https://ieeexplore.ieee.org/document/9643388> (IEEE, 2021).
61. Ba, J., Hinton, G. E., Mnih, V., Leibo, J. Z. & Ionescu, C. Using fast weights to attend to the recent past. *Adv. Neural Inf. Process. Syst.* **29**, 4331–4339 (2016).
62. Miconi, T., Stanley, K. & Clune, J. Differentiable plasticity: training plastic neural networks with backpropagation. In *International Conference on Machine Learning* (eds. Dy J., Krause A.) 3559–3568 (PMLR, 2018).
63. Rodriguez, H. G., Guo, Q. & Moraitis, T. Short-term plasticity neurons learning to learn and forget. In *International Conference on Machine Learning* (eds. Chaudhuri K. et al.) 18704–18722 (PMLR, 2022).
64. Moraitis, T., Sebastian, A. & Eleftheriou, E. Optimality of short-term synaptic plasticity in modelling certain dynamic environments. Preprint at <https://arxiv.org/abs/2009.06808> (2020).
65. Van-Horenbeke, F. A. & Peer, A. Nilrnn: a neocortex-inspired locally recurrent neural network for unsupervised feature learning in sequential data. *Cogn. Comput.* **15**, 1549–1565 (2023).
66. Limbacher, T., Özdenizci, O. & Legenstein, R. Memory-enriched computation and learning in spiking neural networks through Hebbian plasticity. *IEEE Transactions on Neural Networks and Learning Systems* **36**, 2551–2562 (2025).
67. Gokul, V. & Dubnov, S. Poscuda: position based convolution for unlearnable audio datasets. Preprint at <https://arxiv.org/abs/2401.02135> (2024).
68. Tiwari, V. Mfcc and its applications in speaker recognition. *Int. J. Emerg. Technol.* **1**, 19–22 (2010).
69. Schuster, M. & Paliwal, K. K. Bidirectional recurrent neural networks. *IEEE Trans. Signal Process.* **45**, 2673–2681 (1997).
70. Abreu Araujo, F. et al. Role of non-linear data processing on speech recognition task in the framework of reservoir computing. *Sci. Rep.* **10**, 328 (2020).
71. He, W. et al. Comparing SNNs and RNNs on neuromorphic vision datasets: similarities and differences. *Neural Netw.* **132**, 108–120 (2020).
72. Gutmann, M. & Hyvärinen, A. Noise-contrastive estimation: a new estimation principle for unnormalized statistical models. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics* (eds. Whye T. Y., Titterton M.) 297–304 (JMLR Workshop and Conference Proceedings, 2010).
73. Lee, K. et al. Robust inference via generative classifiers for handling noisy labels. In *International Conference on Machine Learning* (eds. Chaudhuri K., Salakhutdinov R.) 3763–3772 (PMLR, 2019).

74. Fisher, R. A. *Iris*. UCI Machine Learning Repository. <https://doi.org/10.24432/C56C76> (1988).
75. Chen, T., Kornblith, S., Norouzi, M. & Hinton, G. A simple framework for contrastive learning of visual representations. In *International Conference on Machine Learning* (eds. Daumé III H., Singh A.) 1597–1607 (PMLR, 2020).
76. He, K., Fan, H., Wu, Y., Xie, S. & Girshick, R. Momentum contrast for unsupervised visual representation learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* 9726–9735 <https://ieeexplore.ieee.org/document/9157636> (IEEE, 2020).
77. Laydevant, J., McMahon, P., Venturelli, D. & Lott, P. A. The benefits of self-supervised learning for training physical neural networks. In *Machine Learning with New Compute Paradigms* <https://openreview.net/forum?id=Fik4cO7FXd> (OpenReview.net, 2023).
78. Tang, M., Yang, Y. & Amit, Y. Biologically plausible training mechanisms for self-supervised learning in deep networks. *Front. Comput. Neurosci.* **16**, 789253 (2022).
79. Lagani, G., Falchi, F., Gennaro, C. & Amato, G. Comparing the performance of Hebbian against backpropagation learning using convolutional neural networks. *Neural Comput. Appl.* **34**, 6503–6519 (2022).
80. Lagani, G., Gennaro, C., Fassold, H. & Amato, G. Fasthebb: scaling Hebbian training of deep neural networks to imagenet level. In *International Conference on Similarity Search and Applications* (eds. Skopal T. et al.) 251–264 (Springer, 2022).
81. Haider, P. et al. Latent equilibrium: a unified learning theory for arbitrarily fast computation with arbitrarily slow neurons. *Adv. Neural Inf. Process. Syst.* **34**, 17839–17851 (2021).
82. Liu, D., Laydevant, J., Pontlevy, A., Querlioz, D. & Grollier, J. Unsupervised end-to-end training with a self-defined target. *Neuro-morph. Comput. Eng.*, **4**, 044005 (2024).
83. Ellenberger, B. et al. Backpropagation through space, time, and the brain. Preprint at <https://arxiv.org/abs/2403.16933> (2024).
84. Kendall, J., Pantone, R., Manickavasagam, K., Bengio, Y. & Scellier, B. Training end-to-end analog neural networks with equilibrium propagation. Preprint at <https://arxiv.org/abs/2006.01981> (2020).
85. Yi, S.-i, Kendall, J. D., Williams, R. S. & Kumar, S. Activity-difference training of deep neural networks using memristor crossbars. *Nat. Electron.* **6**, 45–51 (2023).
86. Momeni, A. et al. *Training of Physical Neural Networks* <https://arxiv.org/abs/2406.03372> (2024).
87. Huang, L. et al. Normalization techniques in training DNNs: Methodology, analysis and application. *IEEE Trans. Pattern Anal. Mach. Intell.* **45**, 10173–10196 (2023).
88. Chen, X., Liu, D., Laydevant, J. & Grollier, J. *Self-Contrastive Forward-Forward Algorithm*. <https://doi.org/10.5281/zenodo.15526033> (GitHub repository: Self-Contrastive-Forward-Forward, 2025).
89. Bandara, W. G. C., De Melo, C. M. & Patel, V. M. Guarding barlow twins against overfitting with mixed samples. Preprint at <https://arxiv.org/abs/2312.02151> (2023).
90. Ozsoy, S., Hamdan, S., Arik, S., Yuret, D. & Erdogan, A. Self-supervised learning with an information maximization criterion. *Adv. Neural Inf. Process. Syst.* **35**, 35240–35253 (2022).
91. Webster, M. B., Choi, J. & Ahn, C. Learning the connections in direct feedback alignment. <https://openreview.net/forum?id=zgGmA9ZcY> (2020).
92. Robinson, J., Chuang, C.-Y., Sra, S. & Jegelka, S. Contrastive learning with hard negative samples. In *International Conference on Learning Representations (ICLR)* <https://openreview.net/forum?id=CR1XOQOUTH-> (2021).
93. Krotov, D. & Hopfield, J. J. Unsupervised learning by competing hidden units. *Proc. Natl. Acad. Sci.* **116**, 7723–7731 (2019).
94. Grinberg, L., Hopfield, J. & Krotov, D. Local unsupervised learning for image analysis. Presented at the *Neuro AI Workshop*, NeurIPS <https://openreview.net/forum?id=rylq7mF8IS> (2019).

Acknowledgements

This work was supported by the European Research Council advanced grant GrenaDyn (reference: 101020684). The text of the article was partially edited by a large language model (OpenAI ChatGPT). The authors would like to thank D. Querlioz for discussion and invaluable feedback.

Author contributions

X.C. and J.G. devised the study. X.C. performed all the simulations and experiments. X.C., J.G., D.L., and J.L. actively discussed the results at every stage of the study. X.C. and J.G. wrote the initial version of the manuscript. All authors reviewed the manuscript.

Competing interests

The authors declare no competing interests.

Additional information

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s41467-025-61037-0>.

Correspondence and requests for materials should be addressed to Xing Chen or Julie Grollier.

Peer review information *Nature Communications* thanks Ange Tato, and the other, anonymous, reviewer(s) for their contribution to the peer review of this work. A peer review file is available.

Reprints and permissions information is available at <http://www.nature.com/reprints>

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025